

Stacks and Queues at Work: Expression Evaluation

Programming and Data Structures — Week 3, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain precisely why a calculator cannot simply evaluate an expression like $3 + 4 * 2$ by scanning left to right and applying operators as they appear; convert an infix expression to postfix notation using the shunting-yard algorithm, by hand, respecting operator precedence; evaluate a postfix expression using a single stack; and trace both algorithms completely on a real, multi-operator expression, connecting every step back to the stack operations covered in this week's first lecture.

Outline

Today covers the calculator's fundamental problem with reading expressions left to right; the distinction between infix and postfix notation and why operator precedence breaks naive evaluation; evaluating a postfix expression using a single stack, traced completely; converting an infix expression to postfix using the shunting-yard algorithm, also traced completely; combining both algorithms into a working calculator; the MTech-track monotonic stack pattern; an in-class quiz; and a summary.

The calculator's problem

Consider the expression $3 + 4 * 2$. Standard arithmetic order of operations requires multiplication to happen before addition, giving $3 + (4 \times 2) = 11$. A program that simply reads left to right and applies each operator as it encounters it would instead compute $(3 + 4) \times 2 = 14$ — the wrong answer. This is the calculator's core problem: the order in which an expression is *written* and the order in which it must be *evaluated* are genuinely different things, and a correct calculator needs a mechanism that respects this difference rather than reading naively.

Infix vs. postfix notation

Infix notation, the form humans normally write, places each operator between its two operands: $3 + 4 * 2$. Postfix notation, also called Reverse Polish Notation, places each operator immediately *after* its two operands: $3\ 4\ 2\ *\ +$. The remarkable property of postfix is that it contains no ambiguity whatsoever and requires no precedence rules or parentheses to evaluate correctly — the order in which operators appear in the postfix string already *is* the correct evaluation order, which is exactly why it is the form calculators and compilers convert expressions into internally.

Evaluating postfix with a stack

```
def evaluate_postfix(tokens):
    stack = []
    for tok in tokens:
        if tok in "+-*/":
            b = stack.pop()      # second operand - popped FIRST
            a = stack.pop()      # first operand
            if tok == '+': stack.append(a + b)
            elif tok == '-': stack.append(a - b)
            elif tok == '*': stack.append(a * b)
            elif tok == '/': stack.append(a / b)
        else:
            stack.append(float(tok))
    return stack.pop()
```

Evaluating a postfix expression needs only a single stack. Every number token is simply pushed. Every operator token pops the two most recently pushed values, applies the operator, and pushes the result back onto the stack — ready to be used by a later operator, or to be the final answer. Note the subtlety in variable naming: because a stack is LIFO, the value popped *first* is actually the *second* operand of the operation (b), and the value popped *second* is the *first* operand (a) — this ordering matters critically for non-commutative operators like subtraction and division.

Tracing postfix evaluation on $3\ 4\ 2\ *\ +$

```
Token '3': push 3          → stack: [3]
Token '4': push 4          → stack: [3, 4]
Token '2': push 2          → stack: [3, 4, 2]
Token '*': pop 2 (b), pop 4 (a), push 4*2=8 → stack: [3, 8]
Token '+': pop 8 (b), pop 3 (a), push 3+8=11 → stack: [11]
Result: pop → 11
```

Tracing `3 4 2 * +` token by token: the three numbers push in order, giving stack `[3, 4, 2]`. The `*` operator pops 2 and 4 (in that order — $b=2, a=4$), computes $4 \times 2 = 8$, and pushes it, giving `[3, 8]`. The `+` operator then pops 8 and 3, computes $3 + 8 = 11$, and pushes it, leaving the stack as `[11]`. The final pop returns 11 — the correct answer, matching the earlier claim that $3 + (4 \times 2) = 11$, and reached here without ever consulting a precedence rule during evaluation itself, because the postfix ordering already encodes precedence.

Tracing a trickier example with subtraction

This example specifically tests the operand-order subtlety, since subtraction is not commutative. Tracing `10 2 3 - *`: the three numbers push normally to `[10, 2, 3]`. The `-` operator pops 3 as b and 2 as a , computing $a - b = 2 - 3 = -1$, not $3 - 2 = 1$ — getting this order backwards is the single most common bug when first implementing postfix evaluation. The `*` operator then pops -1 and 10, computing $10 \times (-1) = -10$, the final result.

Converting infix to postfix: the shunting-yard algorithm

The shunting-yard algorithm, invented by Edsger Dijkstra and named for the way a railway shunting yard rearranges train cars into a new order, converts infix expressions to postfix using a stack to temporarily hold operators until their correct output position becomes clear. The core rule: an operator is only moved to the output once every operator of higher or equal precedence currently above it on the stack has already been moved to the output first — this is precisely what enforces correct precedence without needing parentheses in the postfix result.

The shunting-yard algorithm, in code

```
PRECEDENCE = {'+': 1, '-': 1, '*': 2, '/': 2}

def infix_to_postfix(tokens):
    output = []
    ops = []
    for tok in tokens:
        if tok not in PRECEDENCE:
            output.append(tok)
        else:
            while (ops and ops[-1] in PRECEDENCE
                  and PRECEDENCE[ops[-1]] >= PRECEDENCE[tok]):
                output.append(ops.pop())
            ops.append(tok)
```

```

while ops:
    output.append(ops.pop())
return output

```

Numbers go directly to the output. Operators are compared against whatever currently sits on top of the operator stack: while that top operator has precedence greater than or equal to the incoming one, it is popped straight to output — this is the step that correctly handles cases like $4 * 2$ being fully resolved before a following $+ 3$ is even considered. Once no more higher-or-equal-precedence operators remain on top, the incoming operator is pushed. After the whole input is scanned, any operators remaining on the stack are popped to output in order.

Tracing shunting-yard on $3 + 4 * 2$

```

'3': output=[3]                ops=[]
'+' : ops empty → push         output=[3] ops=[+]
'4' : output=[3,4]             ops=[+]
'*' : top is '+', prec(+)=1 < prec(*)=2 → don't pop, push '*'
      output=[3,4] ops=[+,*]
'2' : output=[3,4,2]           ops=[+,*]
end: pop all → output=[3,4,2,*,+]
```

Tracing the full algorithm on $3 + 4 * 2$: the 3 goes straight to output. The + finds an empty operator stack and is pushed. The 4 goes to output. The * compares itself against the top of the stack, +: since + has *lower* precedence (1) than * (2), the loop condition $\text{prec}(\text{top}) \geq \text{prec}(\text{incoming})$ is false, so + is *not* popped, and * is simply pushed on top. The 2 goes to output. At the end of input, both remaining operators are popped in LIFO order: * first, then +, giving final postfix $3\ 4\ 2\ *\ +$ — exactly the expression evaluated earlier in this lecture, confirming the two algorithms connect correctly end to end.

Tracing a case where precedence forces an early pop

This trace shows the opposite situation: when + arrives and finds * on top of the operator stack, $\text{prec}(\text{*}) = 2 \geq \text{prec}(\text{+}) = 1$ is true, so *is popped to output *before* is pushed. This correctly ensures* $4\ 2\ *$ is fully resolved as a unit before the addition of 3 is applied, producing $4\ 2\ 3\ +$, which evaluates as $(4 \times 2) + 3 = 11$ — the same result a human would compute by hand, and the entire reason the “pop while higher-or-equal precedence” rule exists.

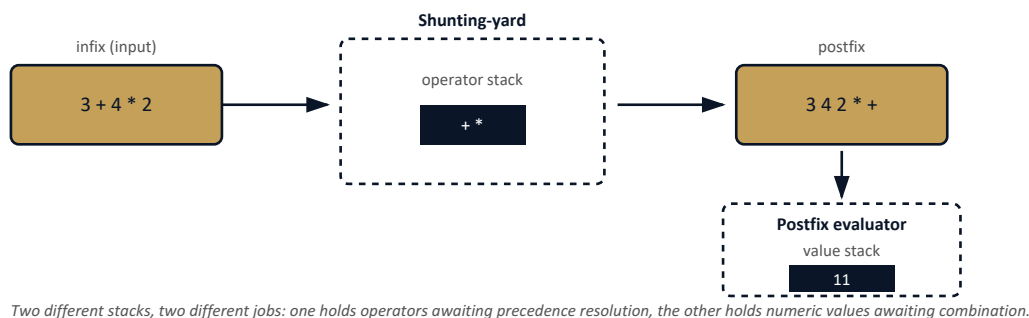
Putting both together: a working calculator

```
def calculate(infix_tokens):
    postfix = infix_to_postfix(infix_tokens)
    return evaluate_postfix(postfix)

print(calculate(['3', '+', '4', '*', '2'])) # 11.0
print(calculate(['10', '-', '2', '*', '3'])) # 4.0
```

Combining both algorithms produces a small but genuinely correct calculator: `infix_to_postfix` handles precedence once, up front, and `evaluate_postfix` then evaluates the resulting unambiguous expression with a simple, precedence-free stack scan. This two-stage pipeline — convert, then evaluate — is exactly the structure real compilers and interpreters use for arithmetic expressions, and both stages have now been traced completely by hand in this lecture, not merely asserted to work.

The picture: two stacks doing two different jobs



It is worth being explicit that this pipeline uses two conceptually different stacks for two different purposes: the shunting-yard stage's stack temporarily holds *operators* awaiting their correct output position, while the evaluation stage's stack holds *numeric values* awaiting combination. Both are genuine stacks in the Lecture 1 sense, applied to different kinds of content.

MTech addition — the monotonic stack pattern

A different but equally important stack application is the monotonic stack pattern, worth knowing as it appears frequently in technical interviews and in real algorithms. Consider the “next greater element” problem: for every element in an array, find the first element to its right that is strictly larger. A naive approach scans rightward from each element separately, costing $O(n^2)$ in the worst case. A monotonic

stack — one deliberately kept in increasing or decreasing order at all times — solves this in a single $O(n)$ pass.

MTech addition — the monotonic stack, in code

```
def next_greater_elements(arr):
    result = [-1] * len(arr)
    stack = []          # indices, kept with decreasing values
    for i, x in enumerate(arr):
        while stack and arr[stack[-1]] < x:
            result[stack.pop()] = x
        stack.append(i)
    return result

print(next_greater_elements([2, 1, 2, 4, 3]))    # [4, 2, 4, -1, -1]
```

The algorithm maintains a stack of *indices* whose corresponding values are kept in decreasing order from bottom to top. For each new element, every index on the stack whose value is smaller than the current element has just found its “next greater element” — the current one — so those indices are popped and resolved immediately. The current index is then pushed. Because each index is pushed exactly once and popped at most once over the entire run, the total work across all n elements is $O(n)$, despite the nested- looking while loop inside a for loop — the same amortized argument style used throughout this course, applied to a new pattern.

Why this matters beyond calculators

This lecture’s algorithms are not toy exercises confined to a classroom. Every compiler and interpreter that parses arithmetic expressions in source code uses fundamentally this same stack-based approach to resolve operator precedence correctly. Spreadsheet software evaluating a formula like `=A1+B2*C3` runs essentially the same shunting-yard-then-postfix-evaluate pipeline internally. The monotonic stack pattern, meanwhile, recurs throughout algorithm interview questions and real applications under different surface descriptions — the stock-span problem, the largest rectangle in a histogram problem, and the “daily temperatures” problem are all, underneath, the identical monotonic stack idea introduced in this lecture.

Exercise

As an exercise, trace `next_greater_elements([2, 1, 2, 4, 3])` by hand, recording the full contents of the index stack after each of the five elements is processed, and confirm it produces `[4,`

2, 4, -1, -1]. Separately, extend `evaluate_postfix` to support the $\wedge$ (exponentiation) operator, being careful that exponentiation is right-associative — $2\wedge 3\wedge 2$ means $2^{(3^2)} = 512$, not $(2^3)^2 = 64$ — a detail the shunting-yard algorithm's precedence comparison must handle specially.

Quiz — evaluate it

Before checking the answer, evaluate the postfix expression `5 1 2 + 4 * + 3 -` by hand, tracing the stack after every token exactly as demonstrated earlier in this lecture.

Quiz — answer

Tracing token by token: 5, 1, 2 push to give [5, 1, 2]. The first + pops 2 and 1, computes $1 + 2 = 3$, giving [5, 3]. 4 pushes, giving [5, 3, 4]. The * pops 4 and 3, computes $3 \times 4 = 12$, giving [5, 12]. The second + pops 12 and 5, computes $5 + 12 = 17$, giving [17]. 3 pushes, giving [17, 3]. The final - pops 3 and 17, computes $17 - 3 = 14$, the final result — matching the infix reading $5 + (1 + 2) \times 4 - 3 = 5 + 12 - 3 = 14$.

Summary

Postfix notation removes all precedence ambiguity, letting a single stack evaluate an expression correctly with one left-to-right pass. The shunting-yard algorithm converts infix to postfix by using a stack to hold operators until precedence rules confirm their correct output position. Combined, these two algorithms form a genuinely correct calculator, and both were traced completely by hand across several examples in this lecture, not merely asserted. The monotonic stack pattern solves an entire class of “next greater/smaller element” problems in $O(n)$ rather than the naive $O(n^2)$, using the same amortized-cost reasoning developed throughout this course. This closes Week 3: stacks, queues, the circular buffer, and their applications are now fully covered. Next week moves to linked lists, which abandon contiguous memory and fixed capacity entirely, in exchange for a genuinely different set of trade-offs.